

A PROJECT REPORT ON

SMART CAREER RECOMMENDATION SYSTEM

An AI-Based Web Application for Predicting Suitable Careers Using Machine Learning

"Discover Your Skills • Choose the Right Career • Build Your Future"

Submitted in partial fulfillment of the requirements for the course of

FUNDAMENTALS OF DATA ANNOTATION USING PYTHON

<https://kushicarrer.pythonanywhere.com/>

Submitted by:

Sanjana

Abstract

Choosing the right career path is one of the most important decisions a student makes, yet it is often guided by incomplete information, peer influence, or guesswork rather than a genuine understanding of one's own skills and aptitudes. The Smart Career Recommendation System addresses this problem by applying supervised Machine Learning to predict a suitable career for a student based on measurable, self-reported attributes: academic performance, subject scores, technical and soft skills, interests, and personality type.

The system is implemented as a full-stack web application. The frontend, built with HTML5, CSS3, and JavaScript, presents an interactive form for data collection and a results page describing the recommended career. The backend, built with Python and Flask, receives the submitted data, encodes it using the same preprocessing pipeline used during training, and passes it to a trained Random Forest Classifier. The model was trained using Pandas and NumPy for data handling and Scikit-learn for preprocessing, training, and evaluation, and is persisted using Pickle so that it can be loaded instantly at runtime without retraining.

On a held-out test set, the trained model achieves an accuracy of approximately 73% across ten balanced career classes, which is evaluated further using a confusion matrix and a per-class precision/recall/F1 report. The resulting system gives students an accessible, data-driven starting point for career exploration, complete with a match percentage, required skills, future scope, and a suggested learning path for their recommended career.

Table of Contents

1. Introduction
 2. objectives
 3. System Requirements
 4. System Architecture
 5. Technology Stack
 6. Dataset Description
 7. Methodology
 8. Model Evaluation
 9. System Implementation
 10. Results And Discussions
 11. Conclusion
 12. Future Scope
- References

1. Introduction

Selecting an appropriate career is a decision that shapes a student's academic focus, professional growth, and long-term satisfaction. Traditionally, career guidance has relied on generic aptitude tests, counselor interviews, or informal advice from family and peers - approaches that do not always account for a student's actual academic strengths, technical skills, and personality traits together.

With the growth of Machine Learning, it has become possible to build data-driven recommendation systems that analyze multiple attributes simultaneously and identify patterns that would be difficult for a human counselor to compute manually. The Smart Career Recommendation System uses this approach: it treats career prediction as a multi-class classification problem, where a trained model maps a student's profile (14 input features) to one of ten career labels.

This report documents the design, dataset, machine learning methodology, system architecture, implementation, and evaluation of the project.

1.1 Problem Statement

Students often struggle to identify a career that matches their academic performance, skills, and interests, due to a lack of accessible, personalized, and data-driven guidance tools. There is a need for a system that can analyze a student's profile holistically and recommend a career path backed by measurable evidence rather than intuition alone.

1.2 Scope of the Project

The system covers ten career paths - Software Engineer, Data Scientist, Doctor, Mechanical Engineer, Civil Engineer, Graphic Designer, Business Analyst, Teacher, Lawyer, and Psychologist - and is designed as a demonstrable academic project rather than a production career-counseling service. It is built to be easily extended with additional careers, features, or a larger real-world dataset.

2. Objectives

The key objectives of this project are:

1. To design and generate a structured dataset of student attributes relevant to career prediction.
2. To preprocess the data, including label encoding of categorical attributes, for use in a Machine Learning model.
3. To train and evaluate a classification model capable of predicting a student's most suitable career.
4. To develop a responsive, user-friendly web interface for data collection and result presentation.
5. To integrate the trained model into a Flask backend for real-time predictions.
6. To present each prediction with supporting information: match percentage, required skills, future scope, and a suggested learning path.

3. System Requirements

3.1 Software Requirements

Component	Requirement
Operating System	Windows / Linux / macOS
Programming Language	Python 3.9+
Web Framework	Flask 3.x
ML Libraries	Pandas, NumPy, Scikit-learn
Frontend	HTML5, CSS3, JavaScript (ES6)
Browser	Any modern browser (Chrome, Edge, Firefox)
IDE / Editor	VS Code / PyCharm / any text editor

Table 3.1: Software Requirements

3.2 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 1.6 GHz or higher
RAM	4 GB (8 GB recommended)
Storage	500 MB free space

Table 3.2: Hardware Requirements

4. System Architecture

The system follows a simple three-layer architecture consisting of a presentation layer (frontend), an application layer (Flask backend), and a machine learning layer (trained model and encoders). The flow of a prediction request is as follows:

7. The student fills the career prediction form on the web page (HTML/CSS/JavaScript).
8. The form data is submitted via an HTTP POST request to the Flask route '/predict'.
9. The Flask backend converts the submitted values into the same encoded numeric format used during training (via the saved LabelEncoders).
10. The encoded feature vector is passed to the trained Random Forest model, which returns a predicted career label and class probabilities.
11. The backend looks up supporting information (required skills, future scope, learning path) for the predicted career and renders the result page.

12. The result page displays the recommended career, match percentage, and guidance information back to the student.

4.1 Architecture Diagram (Textual)

Layer	Components	Responsibility
Presentation Layer	index.html, result.html, style.css, script.js	Collects user input, displays results, handles UI interactivity
Application Layer	app.py (Flask)	Routes requests, encodes input, calls the ML model, renders templates
Machine Learning Layer	model_training.py, career_model.pkl	Trains, evaluates, and persists the Random Forest classifier
Data Layer	career_dataset.csv, career_info.py	Provides training data and static career reference information

Table 4.1: System Layers and Responsibilities

5. Technology Stack

Layer	Technology Used
Frontend	HTML5, CSS3, JavaScript
Backend	Python, Flask
Machine Learning	Pandas, NumPy, Scikit-learn
Model	Random Forest Classifier
Model Persistence	Pickle (.pkl)
Data Format	CSV (career_dataset.csv)

Table 5.1: Technology Stack

6. Dataset Description

The dataset used for training (career_dataset.csv) consists of 1,500 student records, each described by 14 input features and one target label (Career Label). Since a large, labeled, real-world career-prediction dataset is not publicly available, the dataset was synthetically generated using per-career statistical profiles - each career is assigned realistic mean/standard-deviation ranges for scores and skills, plus weighted likelihoods for interest, personality type, and preferred subject. This produces a dataset that is

balanced across classes and exhibits learnable, realistic patterns, similar to how synthetic datasets are commonly used for teaching classification concepts.

6.1 Feature Description

Feature	Type	Description
Gender	Categorical	Male / Female
Age	Numeric	Student age (16-22)
Academic Percentage	Numeric	Overall academic score (0-100)
Maths Score	Numeric	Mathematics subject score (0-100)
Science Score	Numeric	Science subject score (0-100)
English Score	Numeric	English subject score (0-100)
Programming Skill	Numeric	Self-rated programming skill (1-10)
Communication Skill	Numeric	Self-rated communication skill (1-10)
Logical Thinking	Numeric	Self-rated logical reasoning skill (1-10)
Creativity	Numeric	Self-rated creativity (1-10)
Leadership	Numeric	Self-rated leadership skill (1-10)
Interest	Categorical	Technology, Science, Arts, Business, Medicine, Social Work, Design, Mathematics
Personality Type	Categorical	Analytical, Creative, Social, Leadership, Practical, Investigative
Preferred Subject	Categorical	Mathematics, Science, English, Computer Science, Arts, Commerce, Biology, Social Studies
Career Label (Target)	Categorical	One of 10 career classes (see Section 6.2)

Table 6.1: Dataset Feature Description

6.2 Career Classes

The target variable, Career Label, contains ten balanced classes: Software Engineer, Data Scientist, Doctor, Mechanical Engineer, Civil Engineer, Graphic Designer, Business Analyst, Teacher, Lawyer, and Psychologist. Each class contains approximately 130-170 records in the dataset of 1,500 rows, ensuring the model is not biased toward a majority class.

7. Methodology

The machine learning pipeline follows a standard supervised-learning workflow, implemented in `model_training.py`.

7.1 Data Preprocessing

The dataset is loaded using Pandas, and rows with missing values are dropped to ensure clean input for training. Numeric features (scores and skill ratings) are used directly, since Random Forests do not require feature scaling.

7.2 Label Encoding

The four categorical input columns (Gender, Interest, Personality Type, Preferred Subject) and the target column (Career Label) are transformed into numeric form using Scikit-learn's `LabelEncoder`. A separate encoder is fit for each column, and all encoders are stored together with the trained model so that the exact same encoding scheme can be reused when new data arrives from the web form at prediction time.

7.3 Train-Test Split

The dataset is split into training and testing subsets using an 80/20 ratio via Scikit-learn's `train_test_split`, with stratification on the target column to preserve class balance in both subsets, and a fixed `random_state` (42) for reproducibility.

7.4 Model Training - Random Forest Classifier

A `RandomForestClassifier` is trained with 200 decision trees (`n_estimators=200`) and a maximum depth of 12, using all available processor cores (`n_jobs=-1`). Random Forest was chosen over a single Decision Tree because it aggregates predictions from many de-correlated trees, which reduces overfitting and generally improves generalization accuracy on tabular data such as this dataset.

7.5 Saving the Model

After training, the model, all label encoders, the ordered list of feature columns, and the computed test accuracy are bundled into a single Python dictionary and serialized to `career_model.pkl` using the pickle module. `app.py` loads this single file at startup, so the trained model does not need to be retrained every time the server runs.

8. Model Evaluation

8.1 Accuracy Score

On the held-out test set of 300 records (20% of the dataset), the trained Random Forest Classifier achieved an overall accuracy of approximately 73%. This is a reasonable result for a 10-class

classification problem with overlapping career profiles (for example, Software Engineer and Data Scientist share several overlapping traits), and reflects realistic, non-trivial classification difficulty rather than an artificially perfect score.

8.2 Confusion Matrix

The confusion matrix below shows the number of correct and incorrect predictions for each of the ten career classes on the test set. Rows represent the true label and columns represent the predicted label, in the order: Business Analyst (BA), Civil Engineer (CE), Data Scientist (DS), Doctor (Dr), Graphic Designer (GD), Lawyer (Law), Mechanical Engineer (ME), Psychologist (Ps), Software Engineer (SE), Teacher (Te).

True \ Pred	BA	CE	DS	Dr	GD	Law	ME	Ps	SE	Te
BA	24	4	0	0	0	1	0	0	1	2
CE	0	23	0	0	0	0	6	1	0	1
DS	0	0	23	0	0	0	4	0	2	0
Dr	0	2	0	27	0	0	0	0	0	2
GD	0	0	0	0	24	0	0	0	0	1
Law	1	0	0	0	0	21	0	2	0	7
ME	0	12	2	0	0	0	14	0	0	0
Ps	2	1	0	1	1	4	1	16	0	4
SE	1	0	1	1	0	0	2	0	24	0
Te	4	0	0	0	1	4	0	2	0	23

Table 8.1: Confusion Matrix on the Test Set (300 samples)

8.3 Precision, Recall, and F1-Score

The table below reports precision, recall, and F1-score for each career class, along with the number of test samples (support) belonging to that class.

Career	Precision	Recall	F1-Score	Support
Business Analyst	0.75	0.75	0.75	32
Civil Engineer	0.55	0.74	0.63	31
Data Scientist	0.88	0.79	0.84	29
Doctor	0.93	0.87	0.90	31

Career	Precision	Recall	F1-Score	Support
Graphic Designer	0.92	0.96	0.94	25
Lawyer	0.70	0.68	0.69	31
Mechanical Engineer	0.52	0.50	0.51	28
Psychologist	0.76	0.53	0.63	30
Software Engineer	0.89	0.83	0.86	29
Teacher	0.57	0.68	0.62	34

Table 8.2: Classification Report (per-class metrics)

The overall weighted average precision, recall, and F1-score are 0.74, 0.73, and 0.73 respectively. Careers with highly distinctive skill/interest profiles - such as Graphic Designer, Doctor, and Data Scientist - achieve the highest F1-scores, while careers with more overlapping traits, such as Mechanical Engineer and Civil Engineer (both technical, mathematics-oriented careers), show more confusion with each other, which is consistent with real-world overlap between these professions.

9. System Implementation

9.1 Frontend Implementation

The frontend is implemented as a single-page website (templates/index.html) containing the Home, About, Features, Career Prediction Form, and Contact sections, plus a dedicated result page (templates/result.html) for displaying predictions. The design uses a blue, purple, and cyan color theme with card-based layouts, smooth hover transitions, and scroll-reveal animations, implemented in static/style.css and static/script.js. The layout is fully responsive, adapting to mobile, tablet, and desktop screen sizes.

9.2 Backend Implementation

The backend (app.py) is a Flask application that loads the trained model bundle on startup (training it automatically if career_model.pkl is not present), serves the home page with form dropdown options generated directly from the saved label encoders, and exposes two POST routes:

- /predict - reads form data, encodes it using the saved encoders, runs it through the Random Forest model, computes the match percentage from predict_proba, and renders the result page with the top 3 career matches.
- /contact - accepts the contact form submission and displays a confirmation message (logged to the console for this academic demo).

9.3 Folder Structure

Path	Purpose
app.py	Flask application and prediction routes
model_training.py	Model training, evaluation, and pickling
generate_dataset.py	Synthetic dataset generation
career_info.py	Required skills / future scope / learning path per career
career_dataset.csv	Training dataset (1,500 records)
career_model.pkl	Saved trained model and encoders
requirements.txt	Python dependencies
templates/index.html	Home, About, Features, Form, Contact
templates/result.html	Prediction result page
static/style.css	Stylesheet (blue/purple/cyan theme)
static/script.js	Client-side interactivity

Table 9.1: Project Folder Structure

10. Results and Discussion

The completed system successfully accepts a student's academic, skill, interest, and personality data through a web form and returns a predicted career along with a match percentage, required skills, future scope, and a suggested learning path. Manual testing with a range of representative student profiles - for example, a student with high programming and logical-thinking scores and a Technology interest was correctly recommended a Software Engineer or Data Scientist career - confirms that the model's predictions align with intuitive expectations.

The model's overall test accuracy of ~73% and the per-class metrics in Section 8 indicate that the system performs well for careers with distinctive feature profiles, and reasonably well for careers that naturally overlap in required skills. This is an expected and realistic outcome, since career fit in the real world is rarely a hard boundary between professions.

11. Conclusion

The Smart Career Recommendation System demonstrates a complete, end-to-end application of Machine Learning to a practical student-guidance problem, spanning dataset design, preprocessing, model training and evaluation, and a full-stack web interface. By combining a Random Forest Classifier with an accessible web form, the system gives students a quick, data-driven starting point for exploring careers that fit their academic strengths, technical and soft skills, interests, and personality.

The project also serves as a practical demonstration of core Machine Learning and web-development concepts - data preprocessing, label encoding, train-test splitting, ensemble classification, model evaluation, model persistence with Pickle, and Flask-based deployment - making it well suited as an academic final-year project.

12. Future Scope

- Train the model on a larger, real-world survey dataset collected from actual students and career outcomes.
- Add more career categories and finer-grained specializations within each career.
- Experiment with additional algorithms (Gradient Boosting, XGBoost, Neural Networks) and compare performance.
- Add user accounts to track predictions over time and measure how recommendations evolve as skills change.
- Integrate real course/certification recommendations (e.g., from online learning platforms) into the suggested learning path.
- Deploy the application to a cloud platform for public access.

References

- [1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [2] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*.
- [3] Harris, C.R. et al. (2020). Array Programming with NumPy. *Nature*, 585, 357-362.
- [4] Flask Documentation - <https://flask.palletsprojects.com/>
- [5] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [6] Scikit-learn Documentation - <https://scikit-learn.org/stable/documentation.html>